



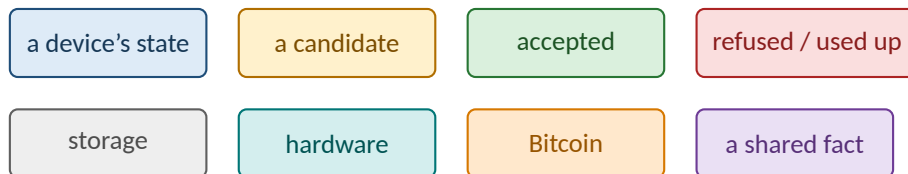
In Plain Language

How Deterministic State Machines, native token emissions, offline payments, Sovereign Finance, and dBTC fit together

A guide for readers who are not engineers

What this is. A plain-language walk through five technical documents: the DSM mathematical explainer, the offline DSM explainer, the SoFi explainer, the dBTC explainer, and the DJTE emissions paper. There are no formulas. Every idea is shown with a picture and a short story about Alice, Bob, and a few friends. Where the original papers name a cost or a limit, this guide names it in the same place.

Colour key used in every diagram



The cast



Alice usually pays; Bob and Carol usually receive. Lena provides liquidity; Tom and Uma trade.

Contents

Part 1	The problem, and the idea	1
1	The problem every digital money has to solve	1
2	The idea, told with a notebook	2
3	Alice pays Bob	3
4	Relationships, not a ledger	3
5	Who decides?	4
Part 2	The hard case, and the boring servers	5
6	Paying two shops with the same money	5
7	Storage that never judges	6
8	What DSM does not promise	7
Part 3	What can be built on it	9
9	Tokens with the rules attached	9
10	Vaults: a box with four doors	10
11	Deals among several people	10
12	Smart commitments, not smart contracts	11
13	Scale, quantum, and proofs	12
Part 4	Native token emissions	13
14	Where new units come from	13
15	Spam without a fee market	14
Part 5	Paying with no internet	16
16	The old answer, and DSM's answer	16
17	The appliance and its three signatures	17
18	The timing trick that keeps one device honest	17
19	Two identities, one device	18

20 Alice tries to pay Bob and Carol offline	19
21 The perfect clone	19
Part 6 SoFi: a market without a middleman	21
22 The vending machine	21
23 One route, one exact price, no slippage	22
24 Two traders, one vault: the one shared decision	22
25 Winning the lock is not the trade	22
26 No front-running	23
27 Prices without an oracle, perpetuals without a clock	24
28 When Lena comes back	24
Part 7 dBTC: Bitcoin that moves like DSM state	25
29 The whole life of a dBTC	25
30 Getting out: the order matters	26
31 Taking out part of it	26
32 “Not your live state, not your coins”	28
33 How dBTC compares	29
Part 8 Limits, and the whole picture	30
34 Where the lines are drawn	30
35 The whole system in one picture	31
36 Short glossary	31

The problem, and the idea

1 The problem every digital money has to solve

When money is just information, the danger is copying. Anyone can duplicate a file, so anyone could duplicate a dollar. Every digital payment system has to answer one question:

IN ONE SENTENCE

What stops the same money from being spent twice?

For fifty years the answer has been a *referee*. A bank keeps the master list. A card network keeps the master list. Bitcoin replaced the bank with a public list that thousands of computers race to extend, and the race decides which of two conflicting payments counts. Faster blockchains change who runs the race and how quickly it finishes. They do not remove it.

In every case, one shared record settles the argument, and every payment in the world has to pass through it, one after another.

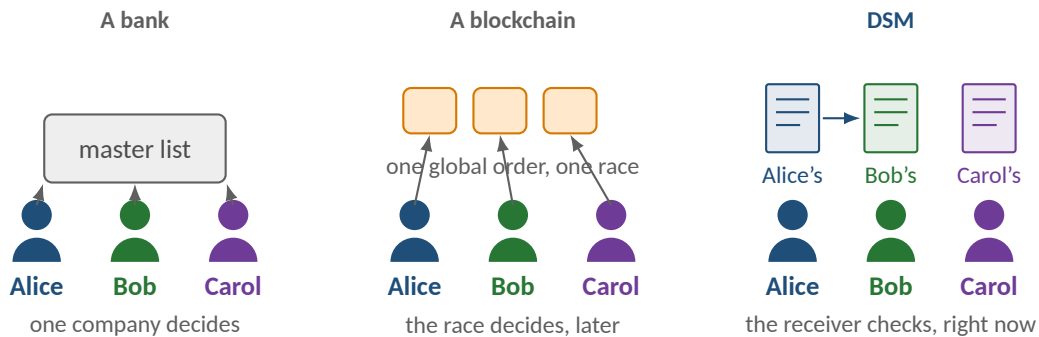


Figure 1: Three ways to stop a double spend. The first two route everything through one shared record. DSM gives every device its own record and lets the person receiving money check it.

DSM — the **Deterministic State Machine** — removes the referee.

Every phone or device keeps its own records. The rule against spending twice is built into those records, in a way the person *receiving* the money can check on the spot from nothing but the bytes in front of them. There is no shared list, no race, no waiting for confirmations, and no company or committee whose approval a payment needs.

The rest of this guide shows how that is possible, what has been built on top of it, and where the design deliberately stops.

2 The idea, told with a notebook

Imagine Alice's money lives in a bound notebook that only she writes in. Each page looks like this:

Alice — page 41fingerprint of page 40: a3f9...

What Alice has right now
50 coins • her standing with Bob, Carol, and Dave

One-time items already used up (crossed out, for good)
✗ ticket #17 ✗ ticket #18

The only pages allowed to come next — and what each one needs
(a) pay Bob 10 — needs Alice's signature, uses ticket #19
(b) pay Carol 10 — needs Alice's signature, uses ticket #19
(c) close the vault with Bob — needs Bob's signature, uses ticket #20

every page carries the fingerprint of the previous one, so the pages form an unbreakable chain

Figure 2: One page of Alice's notebook. The unusual part is the bottom row: the page lists, in advance, the only pages that may follow it.

To pay someone, Alice writes the next page. Three things must be true, and the person she is paying checks all three from a copy of her previous page plus the new one:

1. The new page is **one of the pre-listed next pages**.
2. Its **condition is met** — usually Alice's signature.
3. It **crosses out a one-time item** that is not already crossed out.

If any of those fails, the receiver simply does not accept. Nobody else is asked.

IN ONE SENTENCE

For any one-time item, two conflicting uses of it cannot both exist in one valid notebook. That is a mathematical theorem, not a company policy.

The technical papers call the one-time item a *linear resource* and the crossing-out a *consumption*. The names do not matter; the notebook does.

3 Alice pays Bob

Alice and Bob

Alice has 50 coins on page 41. She pays Bob 10 by writing page 42: it is option (a) from the pre-listed choices, she signs it, and it crosses out ticket #19. Bob checks the three things and accepts.

Now Alice tries to write a *different* page 42 that pays Carol 10 instead — option (b). On its own it looks fine: it is pre-listed, it is signed, and it crosses out ticket #19. But ticket #19 is already crossed out on the page 42 Bob accepted. The two pages cannot both belong to Alice's notebook.

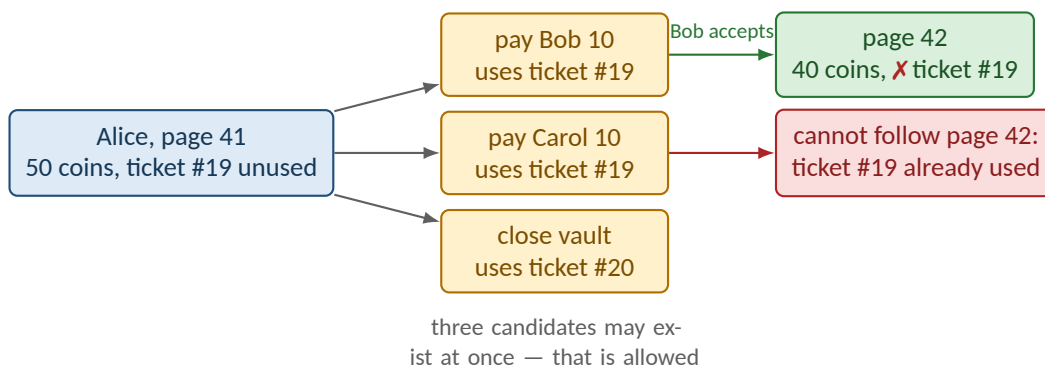


Figure 3: Many next pages may be *proposed*. Only one that uses ticket #19 can ever be *accepted*, because the accepted page crosses the ticket out.

Two words matter here and the papers keep them apart. A page Alice *proposes* is a **candidate**. A page a receiver has checked and folded into the record is **accepted**. Many candidates may exist at once — that is by design, and it is how vaults with several possible outcomes work later. Only one candidate that uses a given ticket can be accepted.

WHY THIS IS NOT A RACE

In Bitcoin, two payments that spend the same coin are each individually valid, and the network's ordering contest decides which one lands. DSM has the same "two valid candidates" moment. What is different is what settles it. There is no contest: a one-time item has exactly one acceptor, and that acceptor's decision is all-or-nothing. Whichever candidate is folded in first has used the ticket by the time the second is looked at. There is never a window in which two conflicting pages are both "accepted but unresolved."

4 Relationships, not a ledger

A DSM device does not keep one running total of everything in the world. It keeps a **separate chain of pages for each relationship**: Alice-and-Bob, Alice-and-Carol, Alice-and-Dave. Each chain moves only when the two people in it agree to move it.

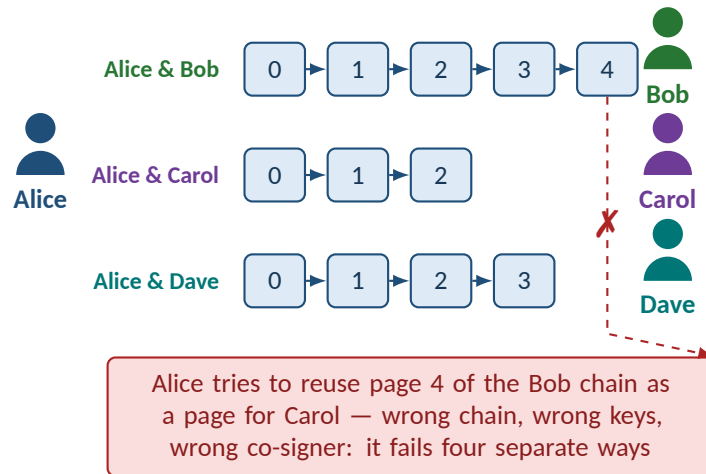


Figure 4: Each relationship is its own chain. A payment made on the Alice–Bob chain names Bob in its addressing, its keys, and its co-signature. Presented anywhere else, it is meaningless bytes.

This is why DSM needs no global order. Two chains that never touch each other never need to be lined up. A million unrelated relationships can move at the same time, and nobody has to sequence them. Where an application really does need “this happens before that,” the ordering is written into the pages themselves — not imposed on the whole world.

5 Who decides?

The person receiving the money. Not a miner, not a validator, not a server.

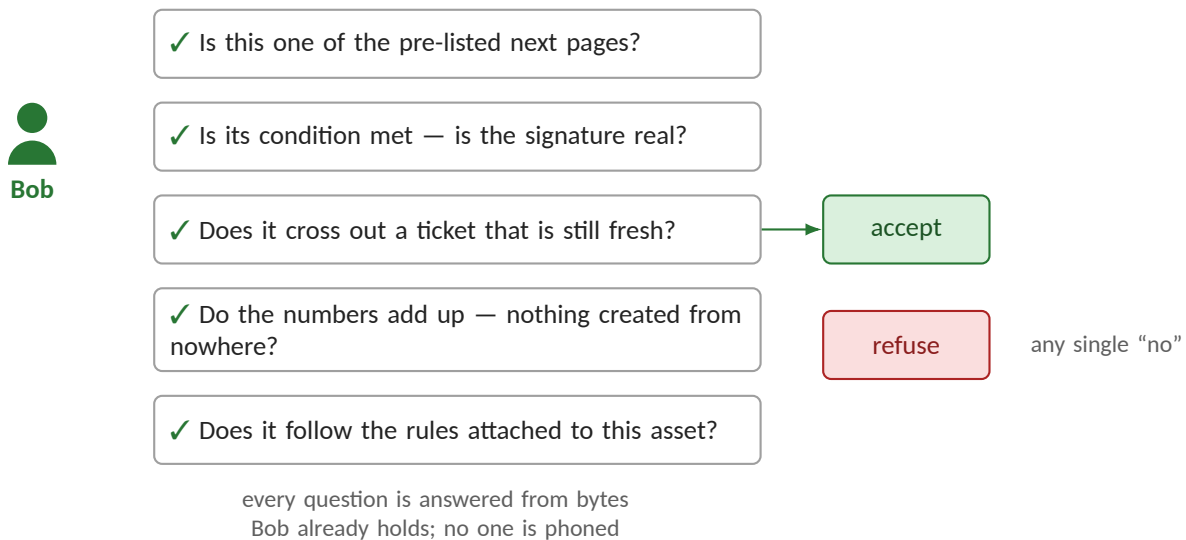


Figure 5: Bob’s checklist. The answers are yes-or-no, and anyone with the same bytes gets the same answers.

Bob recomputes everything from the bytes he holds and gets a yes or a no. Because the check is *deterministic* — the same inputs always give the same answer — no third party is consulted, and no third

party *could* reach a different verdict.

Part 2

The hard case, and the boring servers

6 Paying two shops with the same money

The notebook handles one receiver looking at two pages. But what if Alice shows one page to Bob and a *different* page to Carol, and Bob and Carol never talk? This is the classic offline-merchant fraud: pay two shops with the same coin and hope neither finds out.

DSM handles it with something simple: **numbered lockers**.

Every device has a row of numbered lockers kept by a small group of storage servers. Locker 41 holds the fingerprint of Alice's notebook after her 41st move, locker 42 after her 42nd, and so on. Three rules govern them:

1. **Only Alice can write in Alice's lockers.** The write is signed by her device, and the servers check that before storing anything.
2. **Each locker is written once.** First contents win. Nobody — not even Alice — can overwrite or delete.
3. **The servers never judge.** They do not know what a payment is. They store what is put in and refuse to hold two different things in one locker. That is all.

Alice, Bob, and Carol

Alice pays Bob from page 41. Bob will only treat himself as paid once locker 42 holds the fingerprint of the “paid Bob” page and enough servers confirm it. Alice fills it.

Alice now builds a second page 42 — “paid Carol” — and shows it to Carol as if the Bob payment never happened. Carol already knows Alice is on page 41, because relationships begin by each side fetching the other's committed state. So before anything else, Carol reads locker 42.

When Carol reads the next locker, there are exactly three possibilities:

- **It holds something that is not this payment.** Alice has already moved on. Refuse.
- **It is empty.** Carol accepts only once Alice writes this payment's fingerprint into it and enough servers confirm. If Alice will not, Carol will not.

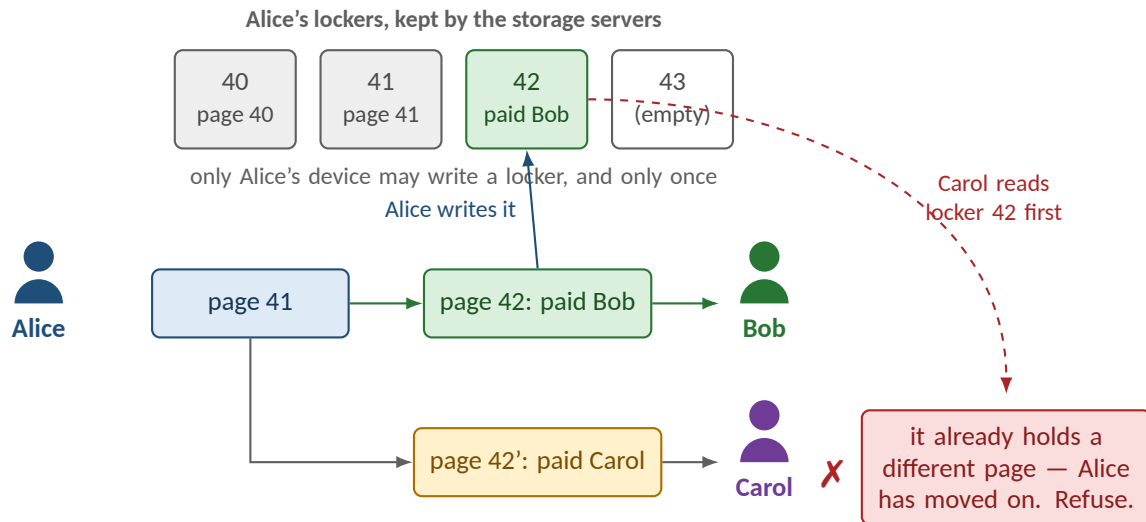


Figure 6: Alice cannot be on page 42 twice. Whatever she shows Carol, locker 42 says what her 42nd move really was.

- **It holds exactly this payment.** Accept.

So either Alice filled locker 42 with the Bob payment — and Carol sees it and refuses — or she never did, in which case Bob never counted himself paid, and Carol's acceptance is what fills the locker. Alice cannot have both. Running two wallets she controls does not help either: the locker belongs to the *paying device*, not to whoever it paid.

IN ONE SENTENCE

Online, a device can name one page per locker, in a locker only it can write and nobody can rewrite. The double spend is not sorted out later by a chain. It is refused at the counter by reading one locker.

WHERE THIS STOPS

Carol needs enough of the storage servers reachable — a majority of a small fixed group. If she cannot reach them she is not a “weaker” online receiver; she is an *offline* receiver, and Part 5 applies instead. The servers are trusted for exactly one thing: never to say two different things about one locker, even after a restart or a restore from backup. That assumption is written down in the papers, not hidden.

7 Storage that never judges

In Bitcoin, the computers that store the chain are the same computers that validate it. Separating those two jobs sounds like a trick, so it is worth being exact about what a DSM storage server is.

There is no leader election and no consensus protocol between servers. What is asked of the storage

What a storage server does

- ✓ keeps copies of everyone's latest pages
- ✓ delivers messages to people who are away
- ✓ keeps the lockers: first write wins, no overwrite, no delete
- ✓ stores bytes under an address computed from the bytes, so it cannot quietly swap them

What it never does

- ✗ sign a payment
- ✗ decide whether a page is valid
- ✗ compute a balance
- ✗ read a clock
- ✗ vote with other servers
- ✗ know what a transaction is

a librarian, not a judge: it can hand you the wrong book, and you will notice, because you check the fingerprint yourself

Figure 7: The storage tier is deliberately dumb. Everything it holds is re-checked by the receiver.

tier is small: keep bytes available, and never say two things about one locker.

A device earns the right to write by paying a small flat fee to **three different storage operators**. That is also what creating an identity costs, which matters for spam and for how new units are issued (Section 14).

8 What DSM does not promise

Two clarifications keep the rest of the guide honest.

It is not a payment channel

A reader who knows Bitcoin will think of Lightning: two parties, a private chain of signed states, nothing on-chain in the ordinary case. The resemblance is real and it stops early. A Lightning channel is off-chain, but its *safety* lives on the chain. If your counterparty broadcasts an old state, you have to notice within a deadline and punish them — which is why “watchtowers” exist.

DSM has no chain to fall back to and no deadlines. A hostile counterparty cannot take anything from you and cannot forge a page you did not sign. What they *can* do is refuse to continue.

	Lightning channel	DSM relationship
Opened by	an on-chain transaction	both sides fetching each other's state
Safety comes from	the global ledger	the used-up ticket itself
Old state	can be broadcast; punished later	is simply not the next page; never accepted
Dispute window	yes, must be watched	none
Watchtower	needed if you are offline	no such role
Hostile counterparty can	force a close	only decline to continue

Safety is not the same as “it will go through”

DSM guarantees that two conflicting things cannot both happen. It does *not* guarantee that anything in particular *will* happen. A counterparty who walks away, a storage group that is unreachable, a condition that is never met — these are **availability** problems, and DSM keeps them out of the safety guarantee on purpose. The papers call the two properties *safety* and *liveness*, and they are different questions.

WHERE THIS STOPS

Lightning offers something DSM does not: a stranger who never held your latest page can still be made whole by the chain. In DSM, the party who holds your latest page is the party who can accept your next one. The lockers of Section 6 are how that latest page reaches a counterparty who was not there for your last move.

Part 3

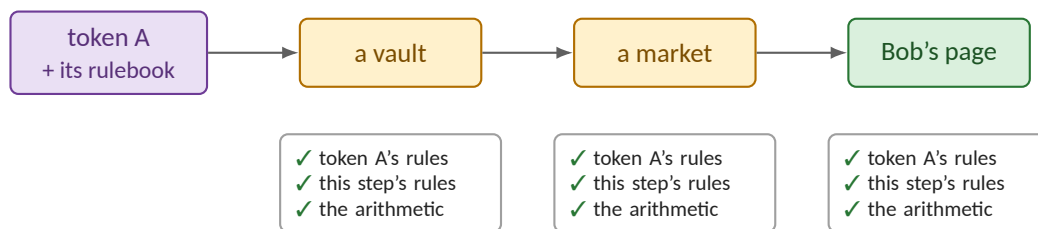
What can be built on it

Everything in this part is the same notebook mechanism — pre-listed next pages, conditions, one-time tickets — pointed at a different purpose.

9 Tokens with the rules attached

On Ethereum a token is a program with an interface. Once you hand the token to another program — a pool, a bridge — *that* program's code decides what happens to it. The token's own rules stop at the boundary.

In DSM a token is an asset with a **committed policy** stapled to it: who may issue it, whether supply is fixed, what a valid transfer needs. Every receiver that ever touches the token checks that policy on every move, including moves inside vaults and markets. The technical name is *CPTA*.



the rulebook is checked at every hop; there is no wrapper that makes it stop applying

Figure 8: A CPTA token carries its rules everywhere it goes. Validity at each step is the *intersection* of the token's rules, the step's rules, and the arithmetic.

What this buys, in plain terms:

- **A fixed supply holds everywhere.** No vault, route, or emission can mint more, because each of those is just another step, checked against the same rules.
- **Rules cannot be bypassed by wrapping.** A market can only *narrow* what the token already allows.
- **Regulated assets can trade in open markets.** A token that says “only holders of credential X may receive this” can sit in a fully public market; only a credentialed trader can produce a valid trade, and the market never needed to know the rule existed.

WHERE THIS STOPS

The flip side is stated plainly in the original: a badly written policy is committed just as firmly as a good one. Cryptography guarantees the policy is enforced faithfully, not that it was designed well.

10 Vaults: a box with four doors

A **vault** (the papers say *Deterministic Limbo Vault*, or DLV) is a box that holds value and has a fixed, pre-listed set of ways it can open — each with its own condition. Because every one of those doors uses the *same* one-time ticket, exactly one door can ever open.

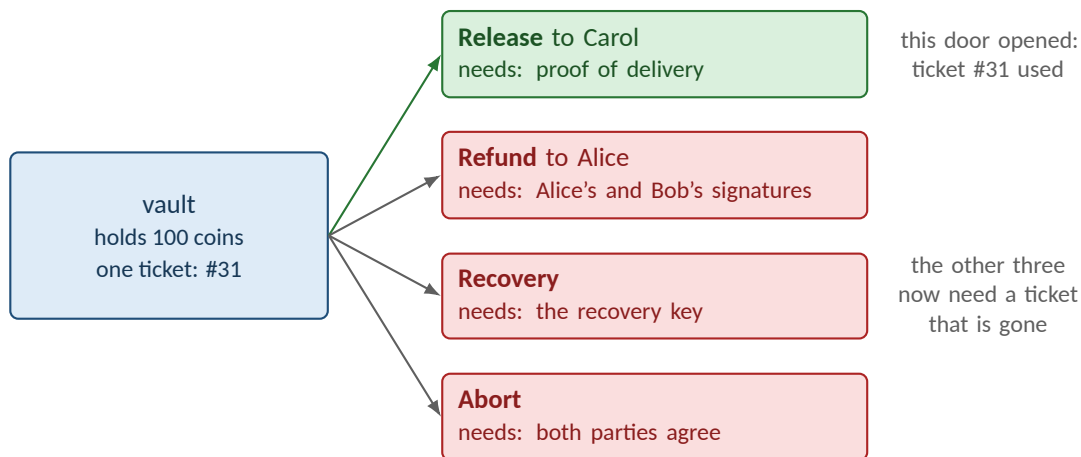


Figure 9: Four doors, one ticket. Whichever door opens first uses the ticket, and the rest are permanently locked — with no chain needed to referee.

This one shape gives you escrow, hash-locked payments, and “time”-locked payments (see Section 12 for what time means here). It is also the shape underneath markets (Part 6) and the Bitcoin bridge (Part 7).

11 Deals among several people

DSM is strictly two-party at its base. So how do three people agree on anything?

They don’t agree on a shared record. They agree on a shared **fact**, written as a fingerprint.

Alice, Bob, and Carol

Alice and Bob will each pay Carol when she delivers a piece of work. There is no three-party ledger and no contract account. There are two vaults and one fingerprint. Alice’s vault (on the Alice–Carol chain) releases when Carol shows a delivery proof with fingerprint P and Alice acknowledges. Bob’s vault (on the Bob–Carol chain) releases on the same P and Bob’s acknowl-

edgement. Carol shows the proof to each of them; each release is an ordinary two-party step.

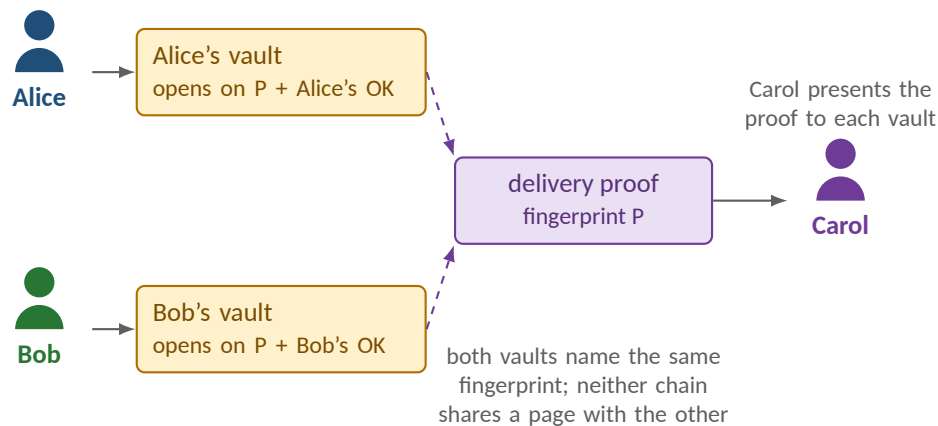


Figure 10: Three people, two chains, one shared fact. If Bob refuses to acknowledge, Alice's vault still releases or refunds on its own terms; Bob's refusal cannot touch Alice's chain.

Overlapping authority does not multiply money. If several signers could each open a different door, the first door opened still uses the one ticket, and the rest fail. Authority changes *which* door is eligible; it never allows two.

WHERE THIS STOPS

All-or-nothing across several people's money is not automatic here. If Alice's leg releases and Bob's does not, that is the outcome, and each vault's refund door is the remedy. Workflows that need true all-or-nothing across several vaults use the one-bundle settlement described in Part 6.

12 Smart commitments, not smart contracts

DSM has no general-purpose contract machine. The papers are direct about why.

Most of what people build on Ethereum is *conditional transfer*: move this value if that condition holds. The unbounded programming language is the delivery mechanism, not the product. DSM keeps the product and drops the mechanism. A **smart commitment** is a bounded check over committed inputs — arithmetic, comparisons, signatures, membership in a fixed list — with a fixed evaluation budget. Every possible outcome can be listed before anyone accepts it.

Ethereum pattern	DSM equivalent
ERC-20 token	token with a committed policy (Section 9)
Multisig wallet	several signers as door conditions (Section 11)
Escrow / hash-locked payment	a vault with a proof-of-delivery door (Section 10)
Airdrop / emission schedule	join-triggered emission from a source vault (Section 13)
Oracle	a shared fact, co-signed by the two parties
AMM / DEX	a vault with a committed market policy (Part 6)
Perpetuals, liquidation	a pre-listed liquidation door (Part 6)
Social recovery	a recovery generation with its own doors
Time-lock	“after N accepted steps” on this chain

What you gain: reentrancy bugs, runaway loops, and “what does this contract actually do?” are structurally impossible.

WHERE THIS STOPS

Two things do not carry across. **No clocks:** where a contract would say “after 3 pm,” a commitment says “after N accepted steps on this chain.” That is weaker for wall-clock deadlines and stronger for certainty — no two verifiers ever disagree about whether it holds. **No open-ended calling between contracts:** a commitment can reference only what was committed when it was created. A workload that truly needs unbounded computation is not a DSM workload.

13 Scale, quantum, and proofs

Three short answers to three common questions.

Does it scale? The DSM explainer’s stress case is a game with a hundred million players. The game keeps running on its own servers at game speed; movement and physics never touch DSM. Only the scarce things — a rare item, a currency, a prize — are DSM assets with policies, held in the players’ own devices. Trading an item is verified by the two players involved and nobody else. Adding a player adds that player’s device and relationships; it does not add their actions to everyone else’s workload, because there is no “everyone else” role.

Is it quantum-safe? The cryptography underneath (BLAKE3 hashing, SPHINCS+ signatures, Kyber key exchange) is chosen to stay secure if large quantum computers arrive. The safety theorem itself does not depend on which primitives are used.

Is it proved? The central rule — that a one-time ticket cannot be used twice in one valid history — is not just asserted; it has been proved with a proof assistant, with no unproven assumptions. That is a different kind of evidence from a track record. Bitcoin’s decade of adversarial uptime is evidence of a kind no proof supplies, and in the paper’s words the honest position is to want both.

Part 4

Native token emissions

Bitcoin issues coins through mining, on a schedule, which needs a chain and a clock. DSM has neither. So it needs a different answer, and the emissions paper gives one — along with an answer to the question that follows it: with no fee market, what stops someone flooding the network?

14 Where new units come from

The mechanism is called *DJTE* — deterministic join-triggered emissions.

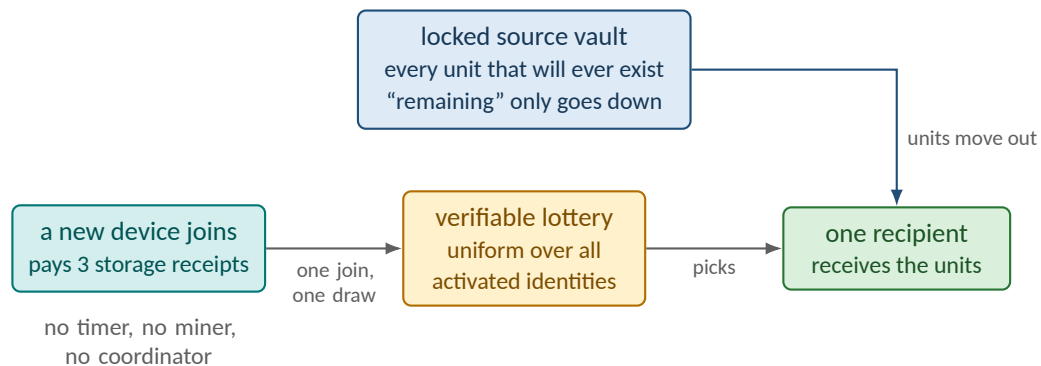


Figure 11: Issuance without mining. A device joining the network is the trigger; a lottery anyone can verify picks the recipient; the vault’s remaining counter is the supply cap.

- **Nothing is minted.** Every unit starts inside a locked vault whose “remaining” number can only decrease. When it hits zero, the vault emits zero forever.
- **Joins are the trigger.** Each new device that pays its storage receipts produces one join proof, which is used exactly once to drive one emission. The rate is set by how many devices join, and nothing else — no timer, no difficulty, no miner.
- **The lottery is verifiable.** The recipient is drawn uniformly across every activated identity, and the draw comes with a proof anyone can check offline. Nobody runs it; there is no timing to manipulate.
- **The schedule is halving-shaped,** over sixteen epochs, and will look familiar.

Dave joins, Carol wins

Dave sets up a new device and pays the joining fee to three storage operators. That produces a one-time join proof. The join proof, the vault’s current fingerprint, and the emission number

are hashed together into a seed nobody could have chosen. The seed picks a number between 1 and N , where N is how many devices have ever joined. That number is walked down a tree of counts — “how many joined with an ID starting with 0? with 1? with 10? ...” — until it lands on exactly one device: Carol’s. The vault sends Carol the units, marks Dave’s join proof as used, and lowers its remaining counter. Dave’s device, Carol’s device, and anyone else who fetches the receipt can re-run every step and get the same answer. Dave had a one-in- N chance of being the winner himself; joining is the trigger, not a ticket you buy.

What happens when views differ

There is no referee here either, so two devices can briefly hold different pictures of the vault — one has seen an emission the other hasn’t. The paper calls this a fork and does not pretend it away. The rule is a pure function of what a device holds: take the state with the highest emission number, and if two tie, the one with the smaller fingerprint. Two devices that have seen the same set of emissions always land on the same answer. Convergence comes from sharing what you’ve seen, not from voting.

WHERE THIS STOPS

Three limits, all in the paper. Uniform-over-identities is only as fair as identity creation is expensive: the lottery is mechanically correct even if someone creates a thousand identities, and whether that is *worth* doing depends on the joining fee costing more than an emission is worth — the spend-gate’s job, not the lottery’s. Emissions follow joins, so they are as smooth or as bursty as adoption is; there is no timer to even them out. And before anyone has joined, an emission is simply a no-op: with nobody to pick, the amount must be zero, and any receipt that names a winner anyway is invalid.

15 Spam without a fee market

A blockchain has one global queue of pending transactions, and anyone can throw things into it that every node has to look at. Congestion, censorship-by-spam, and front-running all live there, and the fix is a fee market run on a clock: pay more to get into the next block. DSM has no queue, no block, and no clock, so it needs a different answer.

Four things, and none of them is a clock. A message must name the receiver’s expected context — their latest page, the expected step number — so random bytes cannot even be represented as a candidate, and are thrown out before any expensive checking. Your device processes an inbox only for contacts you have established, and you can block a contact without touching any other relationship; a blocked sender costs you nothing even while you are online. A step that does nothing is not a step, so “send nothing a million times” cannot be written down. And for the one thing left — someone generating large volumes of *valid* traffic — each identity carries a bundle of 1,000 prepaid credits; every accepted

step you author costs one, and when they run out the only valid move is a refill through the same storage fee you paid to join. *Receiving* never costs anything, so nobody can drain your credits from outside. The number is chosen so that a normal person never notices it and a flooder pays constantly.

Alice blocks a pest

Someone Alice once dealt with starts sending her a stream of junk. Alice's device already ignores anything that doesn't fit her expected next page, so most of it is discarded before any real checking. For the rest, she blocks the contact. From that moment her device stops reading that inbox entirely, while every other relationship carries on untouched. The pest has spent one credit per accepted message the whole time; Alice has spent nothing, because receiving is free.

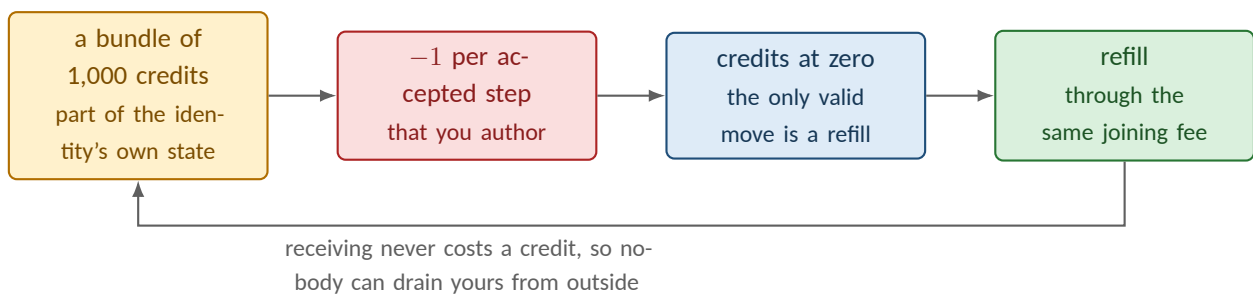


Figure 12: DSM's replacement for a fee market: a fixed bundle of sender-pays credits, refilled at the joining fee, with no clock and no queue.

WHERE THIS STOPS

What a flooder gains in DSM is small and local: pressure on storage and relay capacity from *valid* traffic they paid for. They gain no influence over any ordering, because there is none; no ability to force anyone to process their messages, because inboxes are per contact; and no way to make a victim pay, because receiving is free. There is also nothing to *buy* here: an urgent message cannot outbid a patient one, because every accepted step costs exactly one credit.

Part 5

Paying with no internet

Now remove the network. Alice and Bob are in a room with no connectivity, no storage servers, and nobody else. What stops Alice from walking into the next room and paying the same money to Carol?

16 The old answer, and DSM's answer

Every offline-cash design for the last thirty years — Chaum and Pedersen's "wallet with observer" in 1992, smart-card purses, most offline central-bank digital currency pilots — puts a tamper-resistant chip in the payer's wallet, has the chip co-sign every payment, and trusts it never to sign the same value twice.

The chip is the judge. When the chip is broken, the money is broken.

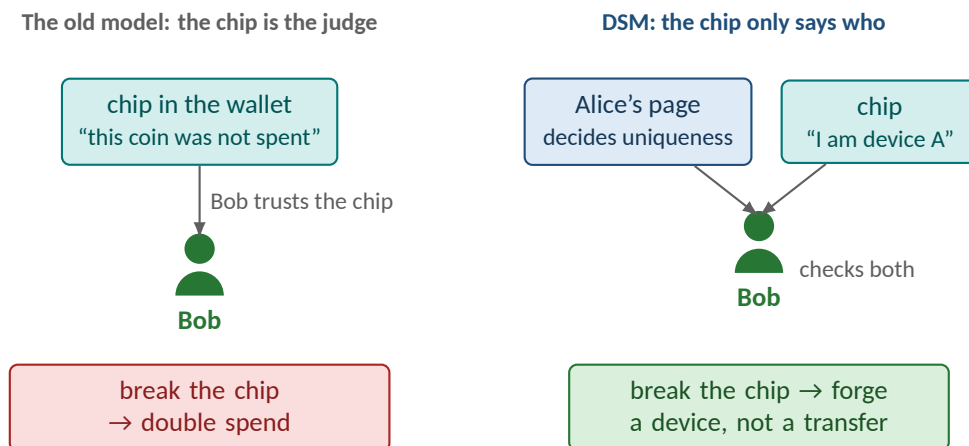


Figure 13: The inversion. In the old model a hardware break is a money break. In DSM, the rule against spending twice never lived in the hardware, so a hardware break lets an attacker impersonate a device — not spend twice.

IN ONE SENTENCE

The software decides; the chip only says who.

Uniqueness offline is the *same* notebook rule as online: Alice's state is one item, a payment uses it up whole, and one page admits one accepted next page. That theorem reads no chip.

The only thing software cannot do is tell the real device from a perfect copy of its files, because a copy runs the same software. That is the one job the hardware is given: **prove that this payment came from the enrolled physical device.**

17 The appliance and its three signatures

Offline DSM uses a phone plus a small piece of hardware: a secure chip that can hold a key without ever giving it up, on a small host board. An offline payment carries **three signatures over the same message**.

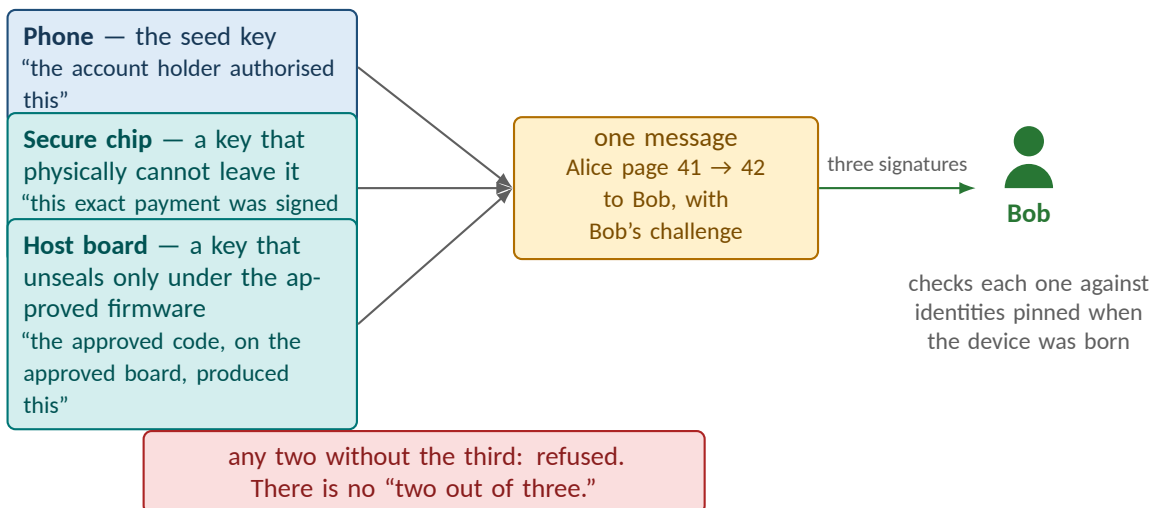


Figure 14: Three keys in three places, one message. The seed proves the holder; the chip proves the die; the host proves the enrolled code ran.

Copying the phone gets you the first signature and nothing else. Reflashing the board with modified firmware gets you the chip's signature but not the host's, because the host key will not unseal for code it does not recognise. What you get is a brick, not a signing machine.

NOTHING LIVE IS CONSULTED

Bob never talks to Alice's chip. He does not open a session, read a counter, or ask the hardware anything. He verifies three signatures over a message he helped build (it includes a random challenge he chose) against public identities fixed at the device's birth. His verdict is computed from bytes he holds, not received from anyone.

18 The timing trick that keeps one device honest

The appliance has a one-way counter that only counts down. The order of operations is what makes it impossible to get two signed payments out of one origin.

So at no moment does a "signed but unspent" payment exist. After the counter ticks, the device's position has moved, and a second payment from the old position fails an arithmetic check before any signature is made.

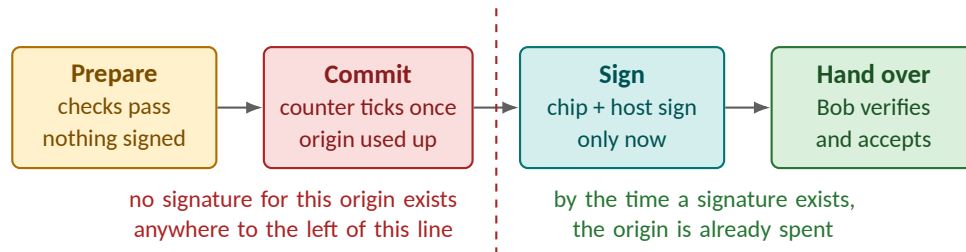


Figure 15: Preparing a payment produces nothing to steal. Committing spends the origin. Only then does the hardware sign.

IN ONE SENTENCE

One appliance, one origin, one release — even to two strangers who will never compare notes. This is a timing property of a one-way counter, not a promise a chip is trusted to keep.

19 Two identities, one device

None of this hardware is required to use DSM.

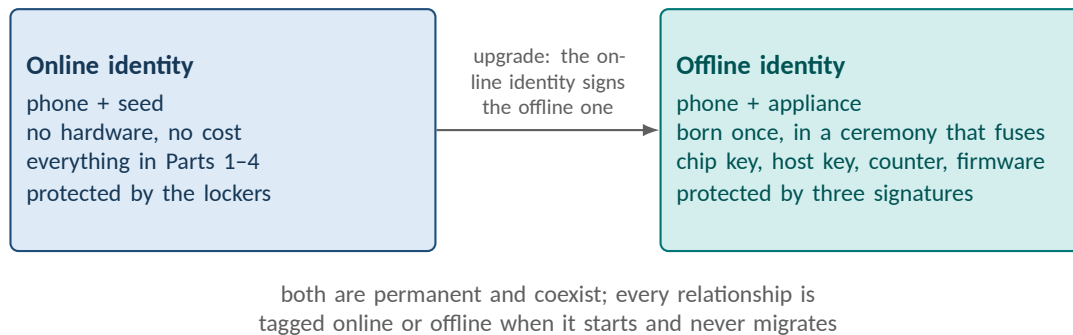


Figure 16: The online identity costs nothing and does everything in the earlier parts. The offline identity is an optional upgrade.

A stolen phone compromises the online identity only; the offline one still lacks the chip and the host. A user who keeps the phone, the appliance, and the seed backup in different places has an offline identity that survives the loss of any one of them, and of any two.

WHERE THIS STOPS

The online identity is deliberately one factor, so a phone compromise is an online-identity compromise. Value that matters should live in offline relationships, or under a policy that requires the offline identity for large movements. And an offline relationship needs a counterparty who has also upgraded.

A spending limit between check-ins

Policy caps how many offline payments a device may make before it reconnects and reconciles with the storage servers. The cap is a *count*, committed when the device is born, so the worst case is a number chosen in advance — not a function of how long the network stays down. A wallet can also earmark a specific amount for offline use, so that losing the device exposes only that slice.

20 Alice tries to pay Bob and Carol offline

Alice, Bob, and Carol, with no network

Alice's appliance is at position 41. She pays Bob: the counter ticks to 42, the chip and host sign, Bob verifies three signatures and accepts. She walks across the room and tries to pay Carol from position 41 again. Carol has no way to ask whether 41 was already used. So the question is not what Carol can see — it is whether Alice can *produce* the second payment at all.

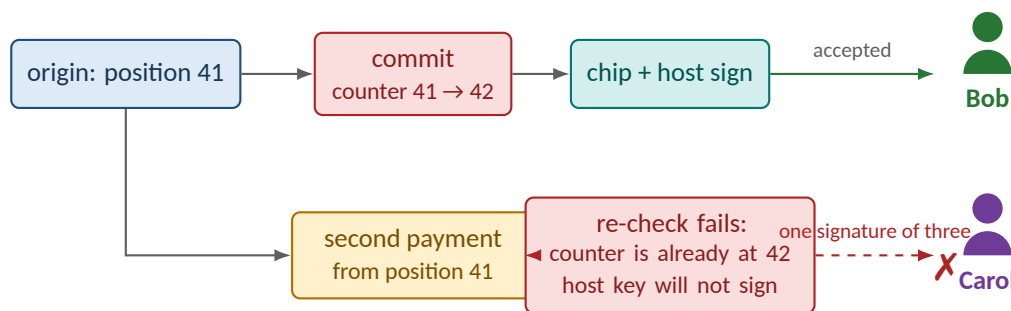


Figure 17: The second payment cannot get its hardware signatures. Alice can sign with her seed — that is software — but a package with one valid signature out of three is refused.

If Carol already knows Alice, it is even simpler: Carol holds Alice's last page, the new package does not follow it, and Carol refuses before reading any hardware at all. That is the notebook rule again.

If Alice runs modified firmware that *would* sign the old position, it is not the approved firmware, so the host key does not unseal. The chip will sign for rogue firmware, but the receiver needs all three.

21 The perfect clone

Take the worst case seriously — and be clear about what it is. It is *not* a copy of the phone: that gets an attacker one signature out of three and nothing else. The worst case is a perfect clone of the **appliance itself**. Suppose an attacker has done the expensive thing: cracked open the secure chip and the host board, extracted the chip's key, the sealed host key, and the counter state, and built a second appliance that is, at the moment of use, indistinguishable from the real one. Both hold position 41. One pays Bob, the other pays Carol. Both packages carry three valid signatures. Both are accepted.

The papers do not pretend to prevent this. No offline-only protocol can tell a perfect clone from the

original — that was true of every smart-card purse and every offline CBDC design too. What is different is *what happens next*.

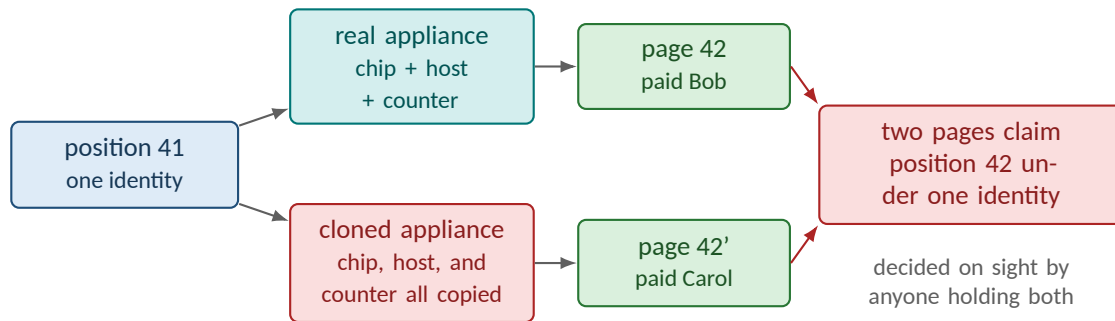


Figure 18: A clone of the whole appliance — not of the phone — produces two payments from one origin, which collide at the same position. The collision is a comparison of two pages, not a suspicion.

When the two packages ever meet — at reconciliation, or through any later transfer that carries their history — the outcome is:

- **They collide.** Two claims to one position under one identity, decidable on sight.
- **They cannot merge.** A receiver holding one branch rejects the other by the notebook rule, without even knowing a split exists.
- **They are attributable.** The evidence names a physical device, not an anonymous key.
- **They are bounded.** The number of such payments between check-ins was capped in advance.
- **They are bricked.** Once the split is seen, both branches are refused until the identity is rotated.

IN ONE SENTENCE

The clone boundary degrades from “one device, one release” to “exposed, bounded, attributable, and bricked.” It does not degrade to silent loss.

WHERE THIS STOPS

Detection is not recovery. Whether a defrauded recipient is made whole is a question for policy and the issuer, not for the state machine. And between the two acceptances and the first comparison, both recipients believe they were paid; that interval is a property of connectivity, not of the protocol.

Part 6

SoFi: a market without a middleman

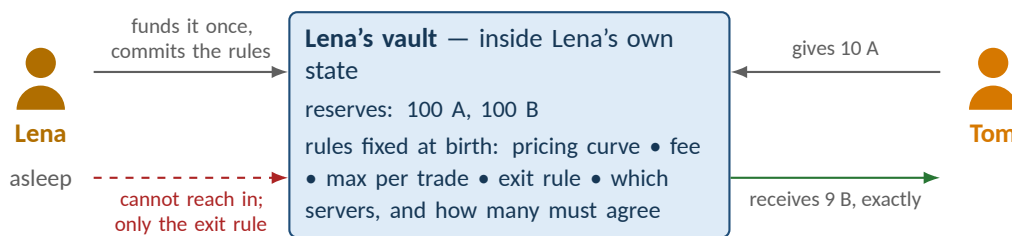
A decentralised exchange is the hardest test of “you can build the applications.” It needs liquidity that trades while its owner is asleep, prices nobody can manipulate, and no operator who can reorder trades. On Ethereum that means a pool contract plus the global transaction order — and an industry built on watching the pending queue to front-run you.

22 The vending machine

Lena and Tom

Lena wants to earn fees by providing liquidity. She stocks a machine with her own goods — say 100 units of token A and 100 of token B — and sets the rules: the pricing curve, the fee, the maximum size of one trade, and the rule for taking her goods back out. Then she goes to sleep.

Tom comes along, puts in 10 A, and the machine hands him the amount of B the rules say. Lena is not consulted. She could not have been; she is asleep. And Lena cannot reach into the machine and pull her goods out except through the exit rule she set when she built it.



no pool, no share token, no contract that can be upgraded or drained — the vault's blast radius is Lena's own reserves

Figure 19: The vault is Lena's own reserves under Lena's own rules. Tom can trade with it because both the value and the permission to exchange it were fixed before he showed up.

Technically the machine is a **vault** (Section 10) inside Lena's own DSM state. Funding it moves value out of her spendable balance into encumbered reserves, in one move, with no second copy anywhere. Getting the reserves back is itself a vault step under the exit rule she committed.

A large trade can draw from several owners' vaults at once, all-or-nothing, without any of them pooling custody. Ten providers can jointly fill a trade none could fill alone.

WHERE THIS STOPS

Lena cannot change the rules of a live vault by a governance vote or an admin call. Changing the rules means closing the vault under its exit rule and opening another.

23 One route, one exact price, no slippage

Tom's own software finds the vault, checks its full history, computes the route, and signs **one exact output**. There is no "accept anything within 1%."

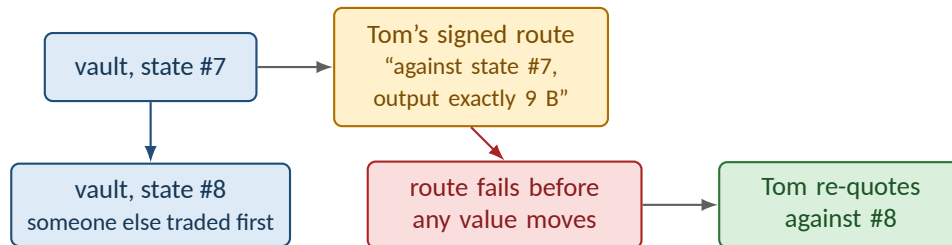


Figure 20: A route is bound to the exact vault state it was priced against. If the vault moved, the trade does not execute at a worse price — it does not execute at all, and Tom gets a fresh quote.

24 Two traders, one vault: the one shared decision

Everything so far is ordinary two-party DSM. A public vault adds one problem that two-party DSM does not have: Tom and Uma can both target the same vault at the same moment, from different places, with no shared history between them. Within one trader's own chain, the notebook rule already refuses a second next page. Across two strangers there is no chain to be linear in.

So SoFi adds one narrow mechanism.

The servers do not parse the trade, do not know the threshold, and do not vote. Whether to call a three-of-five read "consensus" is a question about labels. What matters is that **there is no shared sequencer whose throughput caps the system**: ten thousand vaults trading at once are ten thousand independent reads that never wait for each other.

25 Winning the lock is not the trade

Winning three-of-five locks the vault's current state to one candidate. It does *not* move the reserves.

WHERE THIS STOPS

A trader who wins the lock and never completes has occupied that vault's current state — indefinitely, because SoFi defines no timeout. A timeout would be a clock, and no clock enters validity. The paper states this as the honest price of refusing to let five storage servers move

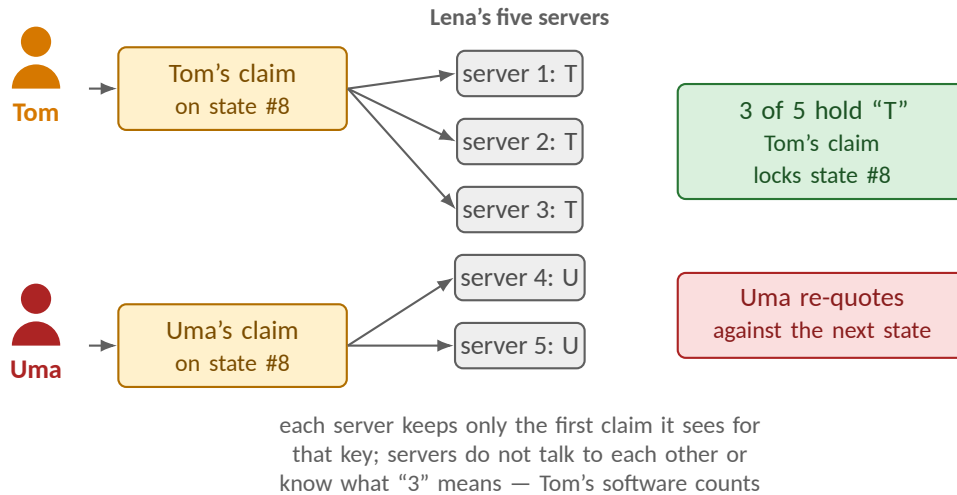


Figure 21: Lena committed five storage servers and a threshold of three when she built the vault. Each server records the first claim it receives for the vault's current state. Whoever's claim three servers hold has the lock.

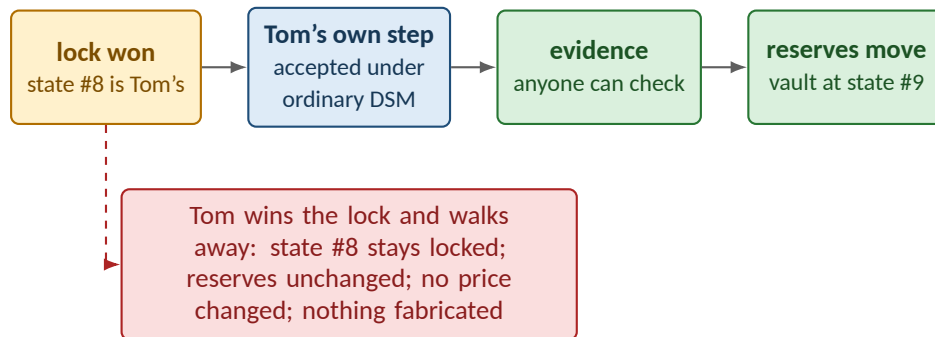


Figure 22: Reserves move only when the trader's own ordinary DSM step is accepted. Storage can lock a state; only an accepted step can spend one.

money. It is an availability cost, not a safety hole: reserves sit at the old state or at exactly the bundled next state, never in between.

26 No front-running

A sandwich attack needs three things: to see your trade before it executes, to insert a trade ahead of it, and to have your trade priced against the state the insertion created. SoFi has none of them.

- There is **no pending queue** to watch. Tom's route is built locally and published to Lena's servers at lock time, not broadcast to a network waiting to order it.
- There is **no priority fee** with which to buy the slot ahead of him.
- The price is **not "whatever the pool is when the chain gets to me."** Tom's output is exact and

bound to a named state.

What an adversary *can* do is race Tom for the same vault and win — at the committed rules' price, for their own trade, with no information about Tom's and no ability to touch his. Tom's trade simply does not happen; he re-quotes.

27 Prices without an oracle, perpetuals without a clock

Reference prices. A reference price is the median of a set of verified, executed trades that the consuming party selects and the counterparty checks. There is no price feed to trust and no block-time window. If the market is too thin to produce a proper set, the price is undefined and anything that depends on it fails closed — by design.

Perpetuals. Funding is charged based on trade activity rather than elapsed hours. Liquidation is a door the position holder signed at open, which a liquidator later proves has been met by presenting the required evidence. No funding timer to game, no keeper race to be first.

28 When Lena comes back

Catch-up is bookkeeping, not approval: the trades were real before she looked. Applying them shortens everyone's history walk for her vault. Closing the vault returns exactly the reserves at the state named in the close, because nobody else can advance that state once the close is locked.

WHERE THIS STOPS

Three boundaries. SoFi trades are **online**: Lena may be away, but Tom needs her servers reachable, and SoFi has no offline mode at all. If a vault's fixed group of servers is permanently gone, that vault cannot be traded or closed. And a close can be blocked by an unresolved trader lock on the current state, for as long as it stays unresolved.

Part 7

dBTC: Bitcoin that moves like DSM state

IN ONE SENTENCE

The dBTC is the asset; the Bitcoin key is machinery.

29 The whole life of a dBTC

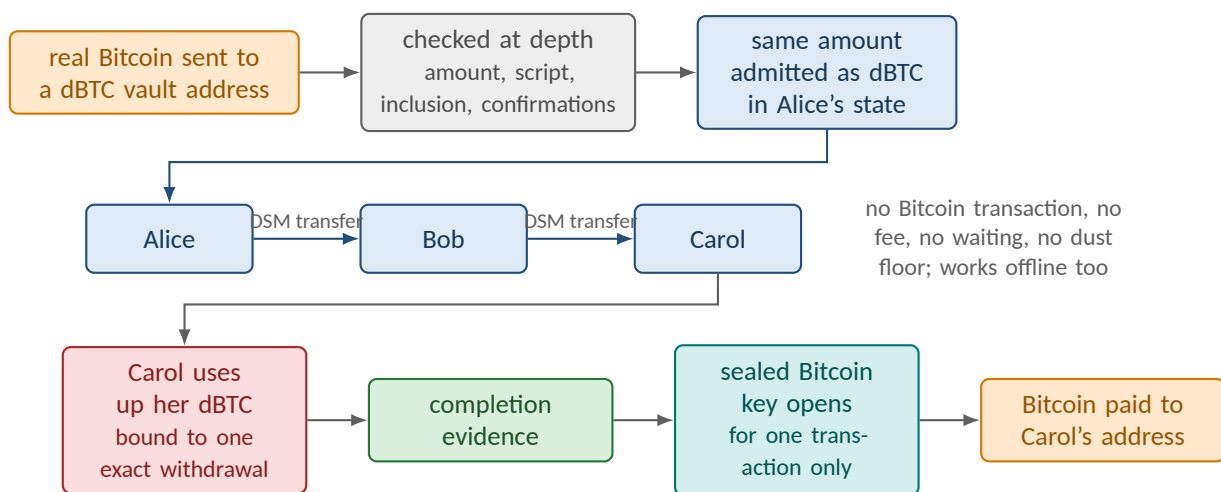


Figure 23: Bitcoin goes in once. The value then moves as ordinary DSM state for as long as anyone likes. Bitcoin comes out once, to whoever holds the dBTC at the end.

Getting in. Someone sends real Bitcoin to an output governed by a dBTC vault script. Before any dBTC exists, the verifier checks the Bitcoin transaction itself: it is valid, the output exists, the amount is exact, the script matches the vault profile, it is included in the chain, and it has enough confirmations. Then exactly that quantity is admitted as dBTC, tied to that origin. No party issues dBTC. Provenance is the deposit.

Moving around. From then on, dBTC moves like every other DSM asset — as ordinary two-party transfers, online or offline. A transfer is not a Bitcoin transaction. There is no fee per hop, no confirmation to wait for, no minimum “dust” amount, and no chain of pre-signed spends to carry. The Bitcoin chain never learns a transfer happened. Bitcoin’s role is collateral and final settlement, not bookkeeping.

30 Getting out: the order matters

The intuitive order is “get the key, spend the Bitcoin, then destroy the dBTC you promised to destroy.” That order requires trusting the promise, and it is how a bridge quietly acquires a custodian. dBTC runs it backwards.

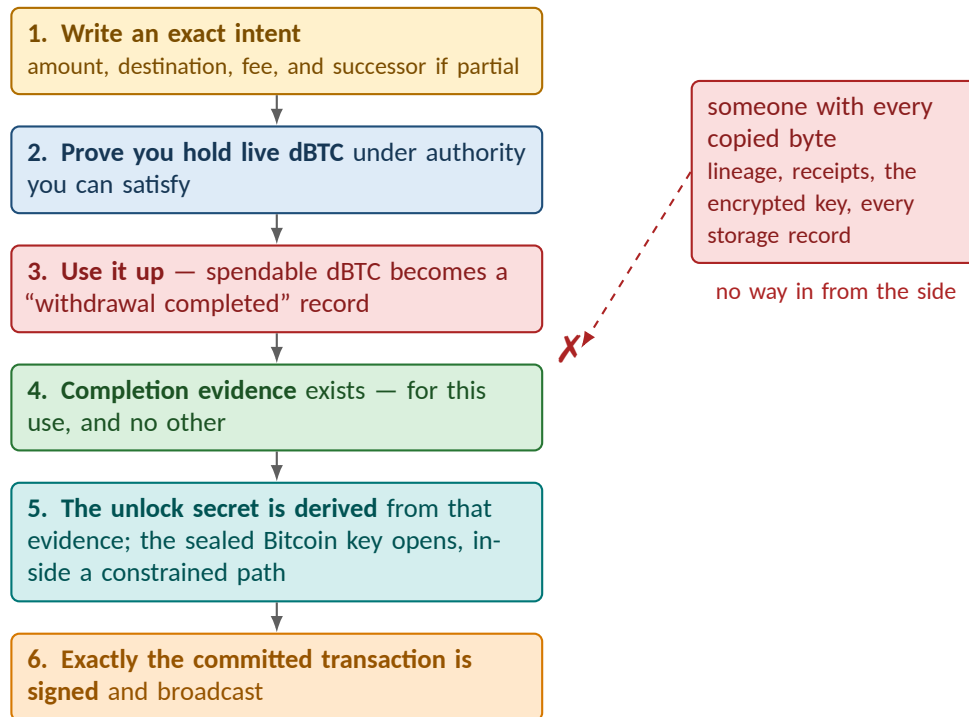


Figure 24: Every step is a consequence of the one above it. There is no path to the signature that does not begin with using up real dBTC.

Think of a safe that opens only after you tear up your claim ticket, where the shredder itself produces the combination — and the combination works for exactly one withdrawal slip you wrote before shredding. A different amount or destination is a different slip, and the key will not sign it.

NO SECOND DOUBLE-SPEND SYSTEM

The bridge introduces no federation, no signing committee, no validator set, no mint, no global dBTC ledger, and no second database of who owns what. DSM has already answered whether the state exists and may be used up; the vault only makes Bitcoin release conditional on that answer.

31 Taking out part of it

If Carol only wants part of her dBTC out, the Bitcoin transaction pays her destination and sends the remainder into a **fresh successor vault**, in the same origin lineage, under a brand-new key.

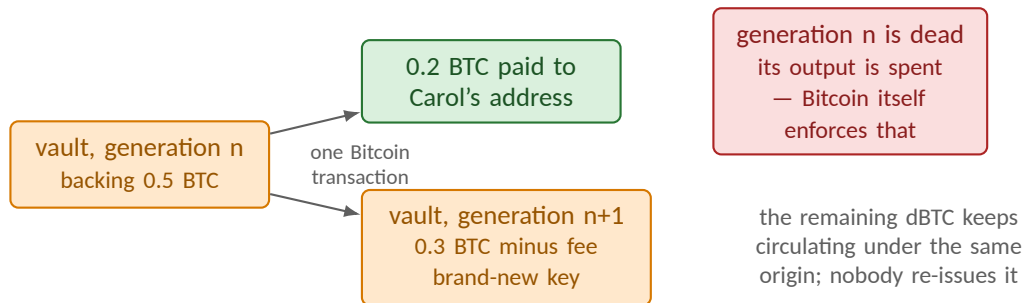


Figure 25: A partial withdrawal is one Bitcoin transaction with no counterparty, no round, and no timelock. The old vault is gone because its output is spent.

Two rules protect everyone *else* who holds dBTC from that origin:

- **The successor can never be left as dust.** A floor is committed when the vault is born and enforced at the moment the dBTC is used up — not as a courtesy check at transaction time. If the split would breach it, the withdrawal is refused outright, and nothing downstream exists to unwind.
- **The remainder is never trimmed into the fee.** It is collateral for dBTC the withdrawer may not own. Destroying it would take value from someone else.

A vault that cannot be partially withdrawn without breaching the floor can still be *fully* withdrawn by whoever holds its remaining dBTC, and a holder whose dBTC spans several origins can draw against whichever active vaults admit the split.

32 “Not your live state, not your coins”

On Bitcoin, knowing the private key is ownership. On dBTC it is not, and the vault is built so that it cannot be.

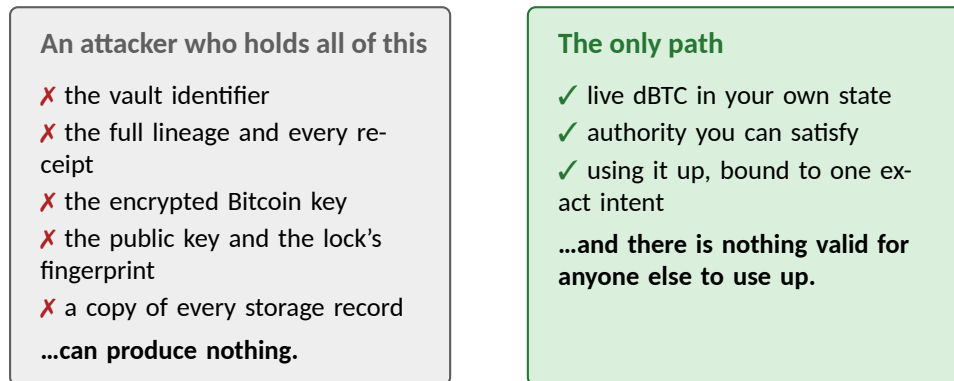


Figure 26: Copying does not steal it, publishing does not weaken it, and storing the vault material in public is safe enough to be the default.

That default is what lets **the depositor disappear**. After admission, the person who funded the Bitcoin has no further role: they sign nothing, approve nothing, and need not exist. A holder years later fetches the vault material from storage, a counterparty, or their own archive, and withdraws on the strength of their own state.

System	Who has to be reachable to get your money out
Lightning	your counterparty, or your watchtower
Ark	the operator, for rounds
Statechains	the entity holding a key share
eCash (Cashu, Fedimint)	the mint
Wrapped Bitcoin	the custodian
dBTC	you

33 How dBTC compares

	Bearer	Offline	Sub-dust	Third party in a payment	What a break costs
eCash / Cashu / Fedimint	✓	✓	✓	the mint	every coin at the mint
Lightning	✗	✗	✓	counterparty, watch-tower	the channel
Statechain	✓	✗	✗	the entity	every coin it touched
Ark	✗	✗	✗	the operator	the tree
Wrapped / federated	✗	✗	✓	custodian	everything
dBTC	✓	✓	✓	none	the holder's own state

WHERE THIS STOPS

Withdrawal is online, pays a real Bitcoin fee, and the successor vault is not usable as backing until it confirms — the cost of touching the chain, paid only by the party who chose to. A Bitcoin reorganisation deeper than the vault's chosen confirmation depth is an external assumption DSM does not claim to prevent. And the inheritance runs both ways: **dBTC is exactly as safe as your DSM custody and no safer.** An attacker who takes over enough of your DSM authority to use up your dBTC has taken it, and the vault will not save you. dBTC also does not offer Chaum-style blinding for privacy. Online and offline dBTC are separate domains with their own proofs; converting between them is an explicit step, and only online dBTC can be SoFi liquidity.

Limits, and the whole picture

34 Where the lines are drawn

The source documents each end with a list of what is deliberately not claimed. Collected here in plain terms.

- **Safety is not liveness.** DSM guarantees that two conflicting things cannot both become real. It does not guarantee progress. A counterparty who refuses to continue, an unreachable storage group, a trader who locks a vault and leaves — these are availability problems by design, and no timeout exists to cut through them, because a timeout would be a clock.
- **Storage is trusted for exactly one thing:** never holding two values for one locker, including across restarts and restores. Everything else it does is content-blind and re-checked by the receiver.
- **Offline, a perfect live clone of every hardware factor is indistinguishable** by any offline-only protocol. DSM converts that from silent loss into a detected, attributed, capped, bricked split. It does not claim to prevent it, and it is why the identity rests on three factors, not one chip.
- **Emissions are not consensus.** Devices that have seen different sets of emissions can briefly hold different pictures; they agree the moment they have seen the same set, because the tie-break rule reads nothing but the set. Fairness of the lottery is bought by the joining fee, not by the lottery.
- **Bitcoin's own assumptions still apply to dBTC:** confirmation depth, reorganisation risk, and reachability to broadcast.

35 The whole system in one picture

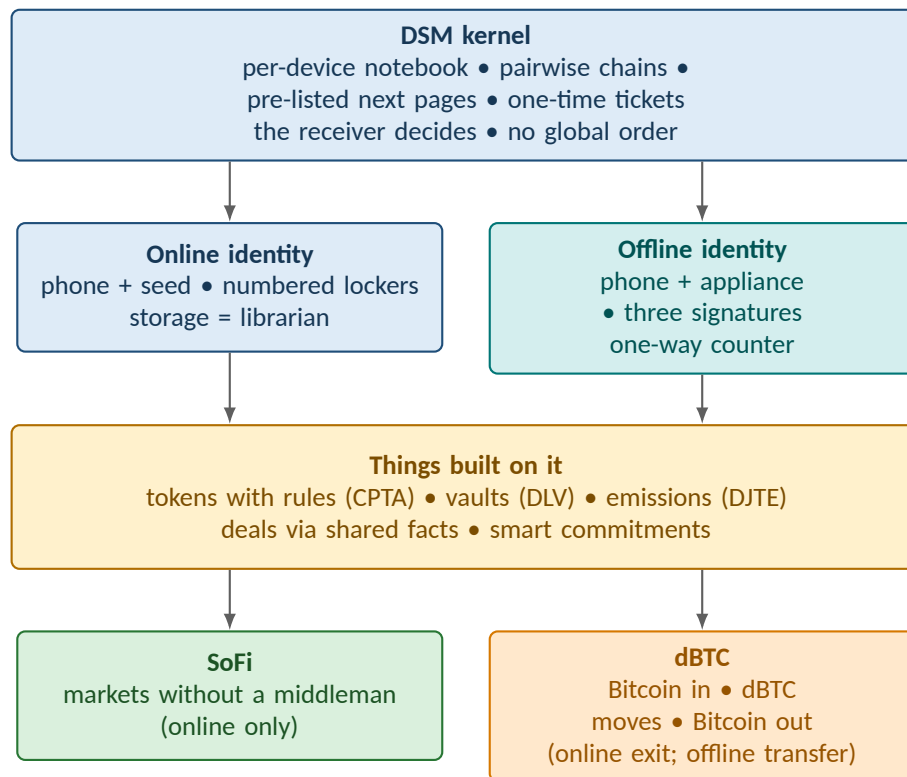


Figure 27: Every layer reuses the same rule.

A token's policy, a vault's doors, a market trade, an offline payment, and a Bitcoin withdrawal are all *one of the pre-listed next pages, meeting its condition, using up a ticket that is not already used, checked by the person who accepts it*. What changes from layer to layer is what the ticket is and who the acceptor is — never who the judge is, because there is not one.

36 Short glossary

DSM (Deterministic State Machine)

The base protocol: each device keeps its own state, any two devices finalise a change directly, and validity is recomputed by the receiver rather than decided by a network.

Linear resource / consumption

A one-time ticket and the act of using it up. The mechanism that makes double-spending impossible to derive.

Tripwire

DSM's name for the rule that two conflicting successors of one used-up ticket cannot both be part of one valid history.

Precommitment

A page lists, in advance, the only pages that may follow it and the condition each one needs.

Lockers (the register)

Write-once cells, one per device per step, kept by storage servers; only that device can write, and only once.

Storage node

A server that stores bytes and keeps the lockers. Never validates, prices, votes, or signs.

CPTA

A token defined as an asset plus a committed policy that every receiver checks on every move.

DLV (Deterministic Limbo Vault)

A vault with a fixed set of doors, of which exactly one can open.

DJTE

DSM's issuance mechanism: a fixed supply in a locked vault, released by device joins to lottery-chosen identities.

Credit bundle

A block of 1,000 prepaid credits each identity carries; every accepted step you author costs one, receiving costs nothing, and a refill goes through the joining fee. DSM's replacement for a fee market, with no clock.

Smart commitment

A bounded, pre-enumerable conditional transfer — DSM's replacement for a general-purpose smart contract.

Appliance / three-factor witness

The offline hardware — a secure chip on a small host board — and the three signatures — seed, chip, and firmware-sealed host — that prove an offline payment came from the enrolled device.

Exposure cap

The committed maximum number of offline payments between check-ins.

SoFi (Sovereign Finance)

The market layer: owner-committed vaults that trade while the owner is absent, with one exact price per trade and a per-vault three-of-five lock as the only shared decision.

Binding vs. realisation

In SoFi, winning the lock on a vault's current state versus the trade actually moving reserves. The

second requires the trader's own accepted DSM step.

dBTC

Bitcoin-backed DSM state. Enters by verified deposit, moves as DSM transfers, exits by using up the dBTC first and then signing exactly one committed Bitcoin transaction.

Successor vault

After a partial withdrawal, the fresh vault holding the remainder under a new key in the same origin lineage.

Safety vs. liveness

"Two conflicting things cannot both happen" versus "something will eventually happen." DSM guarantees the first and treats the second as a separate, explicitly bounded concern.